Mise en place et utilisation de NLL sur SeisComP

Version 2.0 — novembre 2026 — ISTerre - SIG

Table of Contents

Présentation du document 2
Liens et contacts utiles 2
NLL c'est quoi ? 2
Comment ça fonctionne ? 2
Concepts clés à comprendre avant de commencer 3
Le système de coordonnées : 'TRANS' 3
Un modèle de vitesse 1D vs 3D 3
Grille de vitesse en mode 2D radial (spécifique au 1D) 3
Deux grilles différentes à ne pas confondre 3
Profil 3
La voie rapide : Loc_ToolBox 3
Installer et utiliser NonLinLoc sur OAR 4
Etape 1 : Préparer le dossier du profil 4
Etape 2 : Copier le dossier sur OAR 4
Etape 3 : Ajouter NonLinLoc à scolv 5
Etape 4 : Localiser avec scolv 5
Etape 5 : Relocaliser toute une période avec screloc 8
Etape 6 : scrtdd avec les mêmes grilles 8
Comprendre les paramètres 9
Le modèle de vitesse 1D : les lignes LAYER 9
La liste des stations : GTSRCE 9
La grille de vitesse : vel2grid.in 10
Les grilles de temps : grid2time.in 10
Le fichier de contrôle de la localisation : NLL.<profil>.conf 11
CONTROL : niveau de messages et graine 11
TRANS et LOCSIG 11
LOCGRID : la zone de recherche 11
LOCSEARCH : la recherche octree 11
LOCMETH : la méthode et ses seuils 12
LOCGAU, LOCPHASEID, LOCQUAL2ERR, LOCANGLES, LOCTOPO_SURFACE 12
La voie manuelle 12
Ce dont nous avons besoin et qui n'est pas fourni par NonLinLoc 13
Obtenir les algorithmes du package NonLinLoc - Installation 13
Obtention des grilles de temps de trajet avec un modèle 1D 13
Etape 1 Vel2Grid : la grille de vitesse 13
Etape 2 Grid2Time : les grilles de temps de trajets 13
Obtention des grilles de temps de trajet avec le modèle de vitesse 3D de Jean Virieux 13
Etape 1 Variante à Vel2Grid : Mettre le modèle de vitesse au bon format 13
Etape 2 Grid2Time : Calculer les grilles de temps de trajets 14

Présentation du document

Dans ce document vous trouverez le minimum à savoir pour installer et faire fonctionner NonLinLoc sur SeisComP, avec le détail pour le faire fonctionner sur OAR. Nous verrons d'abord les notions communes aux deux utilisations (sur son ordinateur et dans SeisComP), puis le cas de SeisComP.
Depuis la version 1.0, la boîte à outils **Loc_ToolBox** fait l'essentiel du travail : création des grilles, localisation en local, puis préparation d'un dossier prêt à installer sur OAR.
Ce document explique **les concepts**, ce qui ne change pas d'une version des outils à l'autre : ce que fait NonLinLoc, ce que signifient ses paramètres, ce que fait l'installation sur OAR. **Les commandes sont dans le README de Loc_ToolBox**, qui est la référence des outils et qu'on met à jour avec eux ; ce document y renvoie au lieu de les recopier.
Le document présente deux chemins :

- **la voie rapide**, avec Loc_ToolBox (section *La voie rapide : Loc_ToolBox*), recommandée pour toute nouvelle région ;
- **la voie manuelle**, avec les programmes d'Anthony Lomax (section *La voie manuelle*), utile pour comprendre ce que font les outils ou pour un cas qu'ils ne couvrent pas.

Les deux chemins se rejoignent à la partie *Installer et utiliser NonLinLoc sur OAR*. La partie *Comprendre les paramètres* explique, ligne par ligne, les fichiers que lit NonLinLoc et les valeurs choisies par les outils : elle sert dans les deux cas.

Liens et contacts utiles

- http://alomax.free.fr/nlloc/ (site d'Anthony Lomax, documentation de NonLinLoc)
- https://github.com/alomax/NonLinLoc (GitHub de NonLinLoc)
- https://github.com/LuDesmeuzes/Loc_ToolBox (Loc_ToolBox : les outils de ce document ; leur README donne toutes les commandes)
- https://www.seiscomp.de/doc/apps/global_nonlinloc.html (documentation SeisComP de NonLinLoc)
- https://www.seiscomp.de/doc/apps/screloc.html (documentation de screloc)
- Ludovic Desmeuzes (l'auteur de ce document, si vous avez des questions et/ou suggestions de correction/rajout à mettre sur le document) ludovic.desmeuzes@univ-grenoble-alpes.fr

NLL c'est quoi ?

NonLinLoc (Non-Linear Location, NLL) est un ensemble de programmes destinés à la construction de modèles de vitesse, au calcul des temps de propagation et à la localisation probabiliste, non linéaire et globale des séismes dans des structures 3D, ainsi qu'à la visualisation des données volumétriques 3D et des résultats de localisation. (Anthony Lomax : http://alomax.free.fr/nlloc/)
L'algorithme de localisation est implémenté sur SeisComP à travers un plugin (**locnll**). Il faut cependant générer quelques fichiers avec la bibliothèque de programmes mise à disposition par Anthony Lomax : c'est ce que fait Loc_ToolBox.

Comment ça fonctionne ?

NonLinLoc localise des séismes en 3 étapes, chacune gérée par un programme distinct :

- Vel2Grid : modèle de vitesse -> grille de vitesse
- Grid2Time : grille de vitesse -> grilles de temps de trajets (par station et par phase)
- NLLoc : grilles de temps de trajets + pointés -> localisation des évènements

Les deux premières étapes ne sont à faire qu'une seule fois par région et par modèle de vitesse, alors que la dernière est effectuée à chaque localisation d'évènement. Dans SeisComP, c'est le plugin locnll qui joue le rôle de NLLoc : il lui faut donc les grilles de temps de trajets et le fichier de contrôle.
Chaque programme lit un fichier de contrôle (.in). Nous verrons par la suite comment les configurer et les points importants auxquels faire attention.

Concepts clés à comprendre avant de commencer

Le système de coordonnées : 'TRANS'

Dans NonLinLoc, tous les calculs se font en kilomètres dans un repère local centré sur un point d'origine (qu'on définit dans les fichiers de contrôle). La variable 'TRANS' définit ce repère et la façon dont les lat/lon sont converties en kilomètres.
Il en existe plusieurs que vous retrouverez dans la section Control File -> Generic -> TRANS sur le site d'Anthony Lomax. Dans les cas de réseaux compacts de moins de 100 km d'étendue, la 'TRANS SIMPLE' est souvent suffisante. Les outils de Loc_ToolBox utilisent la 'TRANS AZIMUTHAL_EQUIDIST', plus précise, centrée sur le milieu de la zone étudiée (elle m'a aussi permis de résoudre des problèmes de gridding qu'on retrouve assez fréquemment à l'utilisation de NLL).
Règle très importante
La ligne 'TRANS' doit être strictement identique dans tous les fichiers de contrôle ! (vel2grid.in, grid2time.in et nlloc.in, mais aussi dans le fichier de contrôle utilisé par SeisComP que nous verrons à la fin de ce document). profil_vers_seiscomp.py le vérifie avant de préparer l'installation : il s'arrête si le TRANS du fichier de contrôle n'est pas celui des grilles.

Un modèle de vitesse 1D vs 3D

1D : la vitesse ne dépend que de la profondeur. Modèle simple à définir, calcul rapide, c'est pour ce genre de modèle de vitesse que les algorithmes Vel2Grid et Grid2Time nous seront surtout utiles. Pour la partie 3D, Anthony Lomax recommande d'ailleurs (dans la FAQ) de se faire ses propres algorithmes pour générer les grilles de vitesse puis les grilles de temps lors de l'utilisation d'un modèle de vitesse 3D.
3D : la vitesse dépend de x, y et z, calcul plus lourd, mais modèle bien plus précis.

Grille de vitesse en mode 2D radial (spécifique au 1D)

Avec un modèle 1D, il ne faut pas construire une grille 3D complète. NonLinLoc permet de stocker le modèle sous forme de grille 2D (distance, profondeur), bien plus efficace et adaptée à un modèle sans variation latérale.
Ceci se signale en fixant 'xNum = 2' dans 'VGGRID' et en utilisant 'GTMODE GRID2D' dans 'grid2time.in'. Ce n'est pas une vraie dimension X : c'est le drapeau qui active ce mode compact. NLLoc reconvertit ensuite ces temps en position 3D réelle au moment de la localisation. Il n'y a pas de perte de précision, simplement un gain en performance.

Deux grilles différentes à ne pas confondre

'VGGRID', qu'on retrouve dans vel2grid.in, définit la grille du modèle de vitesse (la grille 2D mentionnée juste au-dessus en 1D).
'LOCGRID', qu'on retrouve dans le fichier de contrôle de NLLoc, est la zone de recherche de l'hypocentre. Cette grille doit correspondre à la zone dans laquelle nous nous attendons à trouver les évènements. Elle est indépendante de la représentation du modèle de vitesse, mais doit être entièrement incluse dans la zone pour laquelle vous avez généré les grilles de temps de trajets.

Profil

Sur SeisComP, un **profil** NonLinLoc regroupe un fichier de contrôle et un jeu de grilles de temps de trajets. On en configure un par région et par modèle de vitesse. Loc_ToolBox utilise le même mot : un profil créé en local porte le même nom une fois installé sur OAR. L'usage est de les nommer <pas>_<région>_<1D ou 3D>[_topo], par exemple 05_ubaye_1D_topo pour des grilles au pas de 0,5 km, un modèle 1D et la topographie.

La voie rapide : Loc_ToolBox

Loc_ToolBox enchaîne tout ce que décrit la voie manuelle, depuis le modèle de vitesse jusqu'au dossier à installer sur OAR. Les commandes et leurs options sont dans le README du dépôt ; voici les grandes étapes, et la section du README qui les détaille.

1. **Installer les outils et NonLinLoc**, une seule fois (README, *Installation*). installer_nll.py télécharge et compile NonLinLoc. Le README propose ensuite un premier essai sur le Jura.

2. **Choisir la zone** sur une carte (README, *Choisir la zone sur une carte*) : on dessine le rectangle des séismes à localiser, on règle le modèle, le pas des grilles et la topographie, et la page donne la commande de création du profil.
3. **Créer le profil** avec creer_profil.py (README, *Créer un profil pour une nouvelle région*), à partir d'un modèle 1D (par exemple le modèle à 4 couches du labo) ou du modèle 3D. Le script écrit les fichiers expliqués dans *Comprendre les paramètres*.
4. **Localiser une période en local** avec nll_run.py (README, *Lancer un run*) : les pointés sont récupérés sur le serveur SeisComP, les grilles de temps des stations pointées sont calculées, puis NLLoc est lancé.
5. **Préparer l'installation sur OAR** avec profil_vers_seiscomp.py (section suivante).

Installer et utiliser NonLinLoc sur OAR

Si vous êtes arrivé jusqu'à cette étape, vous avez fait le plus compliqué. Dans cette partie, nous verrons comment localiser des événements via SeisComP : dans scolv, un événement à la fois, ou avec screloc, sur toute une période.
Pour plus de détails, la documentation de SeisComP (https://www.seiscomp.de/doc/apps/global_nonlinloc.html) reste la référence, mais voyons la suite ensemble.

Etape 1 : Préparer le dossier du profil

Sur votre ordinateur, profil_vers_seiscomp.py prépare l'installation d'un profil (commande et options : README, *Installer un profil dans SeisComP*). Par défaut, il vise votre instance OAR. Il ne se connecte à rien et n'écrit rien en base : il remplit le dossier export_seiscomp/<profil>/ du dossier de travail.
Ce dossier contient tout ce dont SeisComP a besoin :

- NLL.<profil>.conf : le fichier de contrôle, avec les chemins d'OAR (y compris celui de la topographie) ;
- topo.asc : la topographie, si le profil en a une ;
- time/ : les grilles de temps de trajets (P et S) des stations pointées dans les runs du profil, et les grilles du modèle ;
- scolv-NLL.cfg et locnll_<profil>.cfg : la configuration de scolv (voir l'étape 3) ;
- screloc.cfg et screloc/entree/ : la configuration de screloc et les pointés de la période des runs (voir l'étape 5) ;
- scrtdd-NLL.cfg et scrtdd_<profil>.cfg : la configuration de scrtdd (voir l'étape 6) ;
- INSTALLER.txt : **toutes les commandes à copier, dans l'ordre**, déjà adaptées à votre profil et à votre machine.

À partir d'ici, il suffit de suivre INSTALLER.txt. Les étapes suivantes expliquent ce que font ses commandes, sans les recopier.
Seules les grilles des stations pointées dans les runs du profil sont copiées : lancez d'abord un run (avec le calcul des grilles) sur la période qui vous intéresse. Le script signale les stations pointées sans grille : NLL ignore leurs phases, sur OAR comme en local.

Etape 2 : Copier le dossier sur OAR

Les deux premières commandes d'INSTALLER.txt créent le dossier des profils sur OAR (~/.seiscomp/nll/) et y copient le dossier du profil.
Sur OAR, on obtient la structure suivante :

```
.seiscomp/nll/
|--- output/                    # fichiers de travail du plugin
|--- 05_ubaye_1D_topo/          # répertoire du profil
    |--- NLL.05_ubaye_1D_topo.conf # fichier de contrôle NLL
    |--- topo.asc               # topographie
    |--- time/                  # grilles de temps de trajets
        |--- layer.P.A194A.time.hdr
        |--- layer.P.A194A.time.buf
        |--- layer.P.A194A.angle.hdr
```

```
    |--- ...
    |--- layer.P.mod.hdr      # grilles du modèle
|--- scolv-NLL.cfg, screloc.cfg, ...
```

Etape 3 : Ajouter NonLinLoc à scolv

Plutôt que de modifier scolv-PROD.cfg, on crée une configuration à part, scolv-NLL.cfg : une copie de la configuration habituelle, à laquelle on ajoute le plugin et le profil. Votre scolv habituel reste ainsi intact.
À la première installation, INSTALLER.txt copie votre configuration habituelle en scolv-NLL.cfg, puis y ajoute les lignes du profil.
Les lignes ajoutées sont les suivantes :

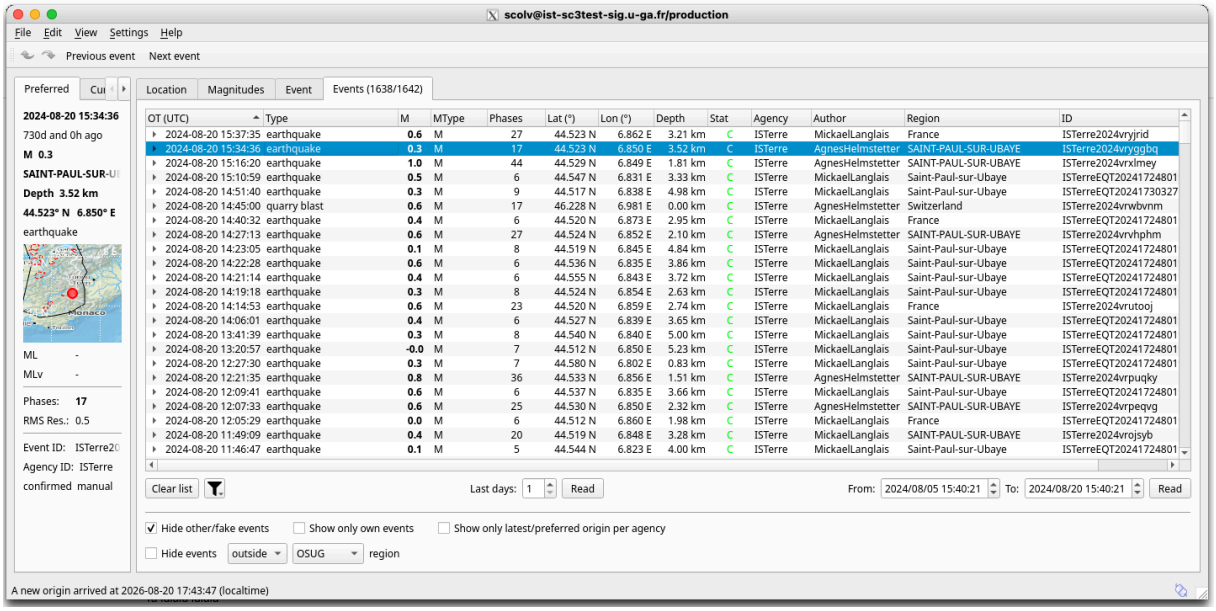
```
plugins = ${plugins}, locnll
NLLROOT = /home/desmeuzl/.seiscomp/nll
NonLinLoc.outputPath = ${NLLROOT}/output/
NonLinLoc.defaultPickError = 0.02
NonLinLoc.profiles = 05_ubaye_1D_topo
NonLinLoc.profile.05_ubaye_1D_topo.earthModelID = 05_ubaye_1D_topo
NonLinLoc.profile.05_ubaye_1D_topo.tablePath = ${NLLROOT}/05_ubaye_1D_topo/time/layer
NonLinLoc.profile.05_ubaye_1D_topo.controlFile =
${NLLROOT}/05_ubaye_1D_topo/NLL.05_ubaye_1D_topo.conf
NonLinLoc.profile.05_ubaye_1D_topo.stationNameFormat = @STA@
NonLinLoc.profile.05_ubaye_1D_topo.region = 44.05, 6.10, 44.96, 7.39
```

- plugins : ajoute le plugin NonLinLoc (locnll) à la liste des plugins chargés.
- NLLROOT : le dossier des profils, repris dans les chemins suivants.
- defaultPickError : l'incertitude (en secondes) donnée aux pointés qui n'en ont pas, c'est-à-dire presque tous. **Elle vaut 0,5 s par défaut dans le plugin** ; on met la même valeur qu'en local (0,02 s), sinon les localisations et surtout les incertitudes diffèrent.
- profiles : la liste des profils disponibles (l'ordre donne la priorité quand scolv choisit un profil tout seul).
- tablePath : le chemin des grilles de temps de trajets, jusqu'au préfixe des fichiers (layer).
- controlFile : le fichier de contrôle du profil.
- region : le rectangle (lat min, lon min, lat max, lon max) où le profil s'applique ; c'est celui de la grille de recherche (LOCGRID).

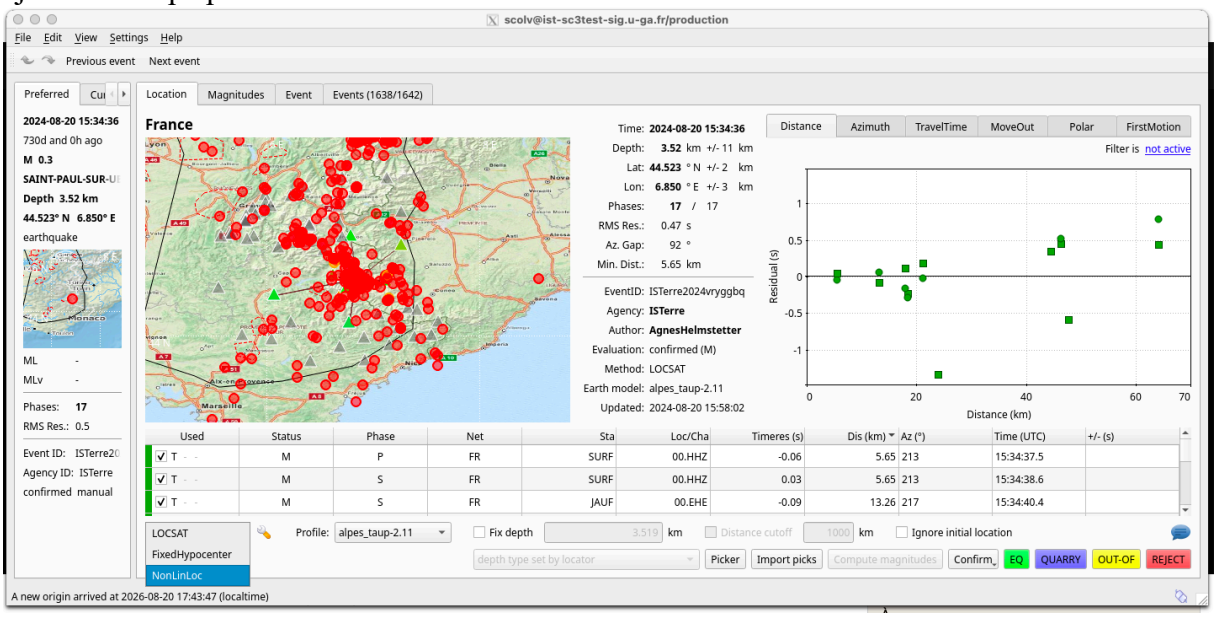
Pour ajouter un deuxième profil à un scolv-NLL.cfg existant, on ajoute seulement les lignes du profil (fichier locnll_<profil>.cfg) et son nom à la liste NonLinLoc.profiles : INSTALLER.txt donne aussi ces commandes.

Etape 4 : Localiser avec scolv

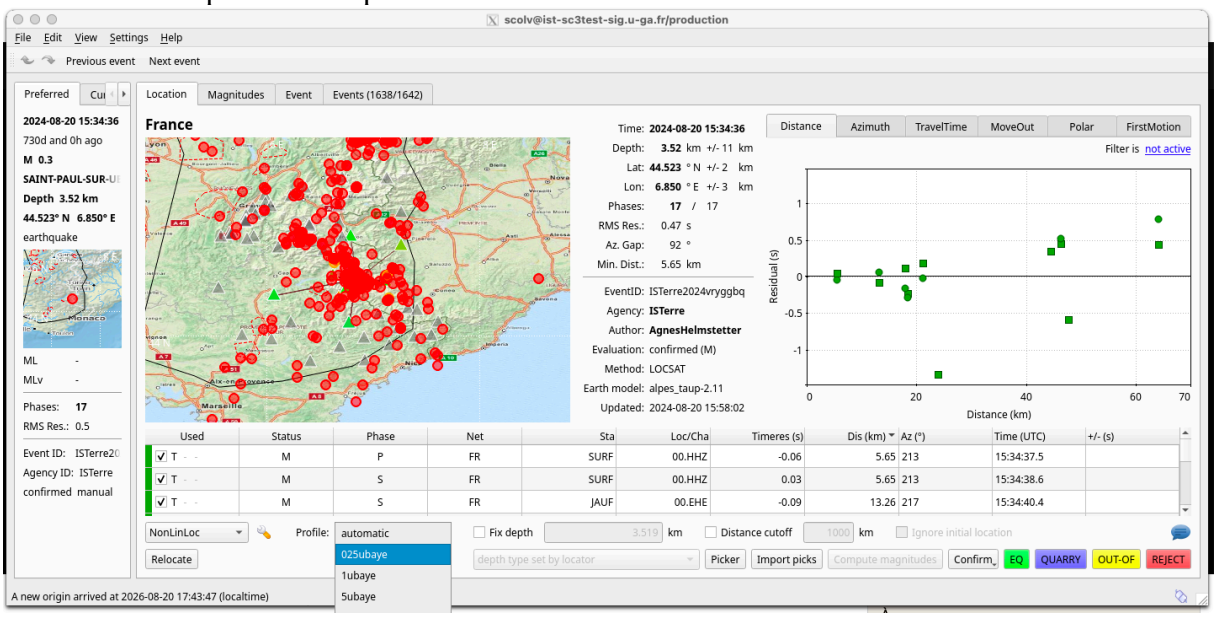
On lance scolv avec cette configuration :
./LaunchScolv.bash NLL
LaunchScolv.bash NLL copie scolv-NLL.cfg en scolv.cfg, lance seiscomp exec scolv, puis supprime scolv.cfg en sortant ; ./LaunchScolv.bash PROD lance toujours votre scolv habituel.
Sélectionnez l'événement que vous souhaitez localiser.



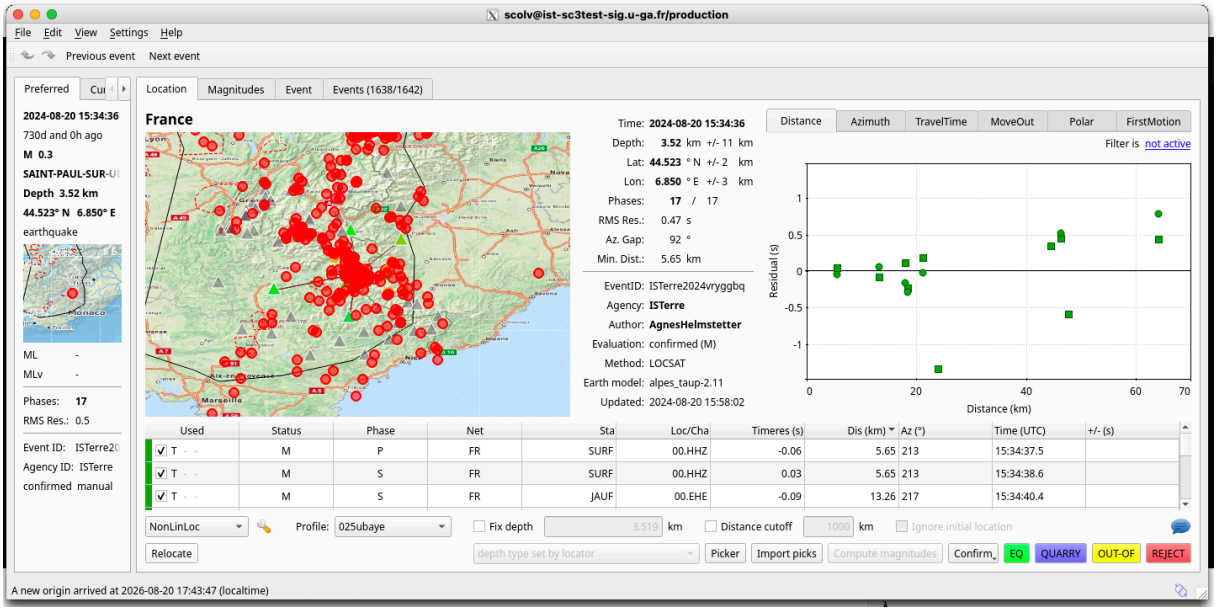
Changez la méthode de localisation et sélectionnez NonLinLoc, qui devrait apparaître si le plugin a bien été ajouté à l'étape précédente.



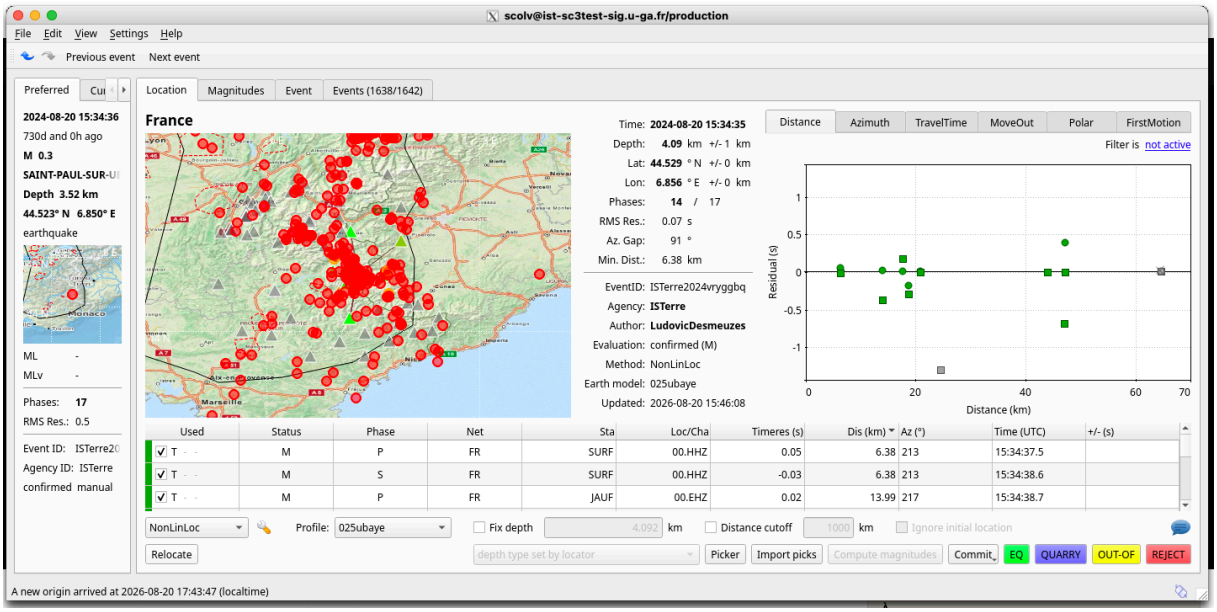
Sélectionnez le profil correspondant à votre étude.



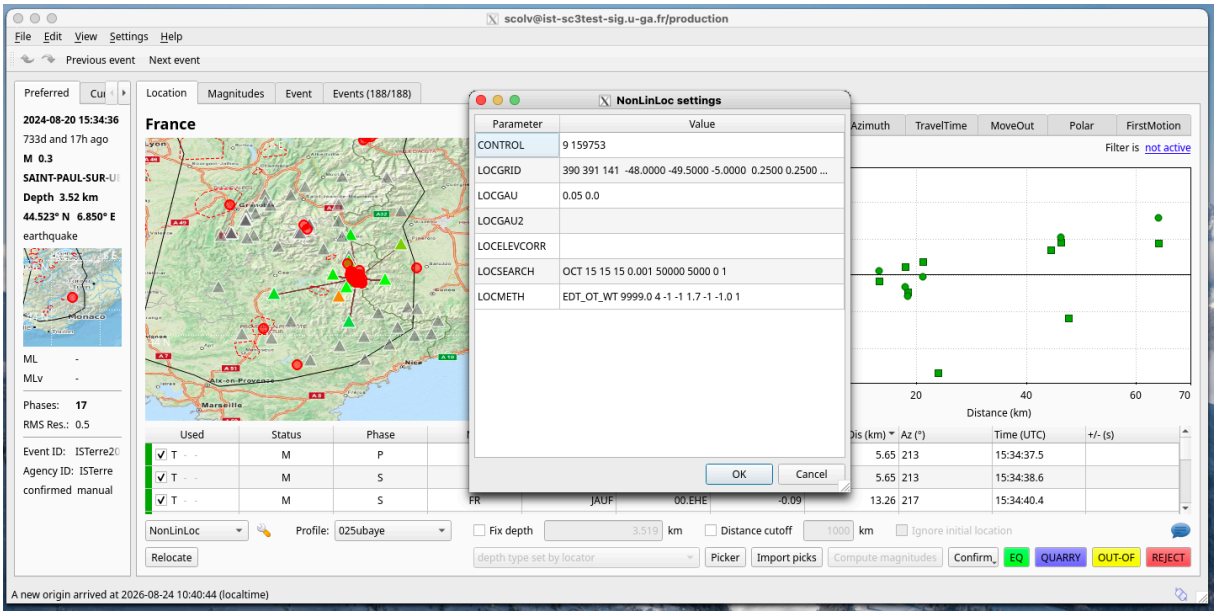
Et cliquez sur Relocate pour lancer NonLinLoc.



Vous voilà maintenant avec une belle et toute nouvelle localisation.



Attention : Relocate ne change rien dans la base. C'est **Commit** qui enregistre l'origine dans la base de production, à la vue de tous : pour des essais, relocalisez et comparez sans commiter.
Il est possible de venir directement modifier les paramètres de NonLinLoc sans avoir à accéder au fichier de contrôle, mais attention ces modifications ne sont pas permanentes et seront écrasées par la configuration du fichier.



Pour gagner du temps avec screloc, on peut baisser le niveau de messages (premier chiffre de CONTROL) : voir *CONTROL : niveau de messages et graine*.

Etape 5 : Relocaliser toute une période avec screloc

screloc relocalise sans interface toutes les origines d'un fichier XML (option --ep). profil_vers_seiscomp.py joint au dossier les XML journaliers des pointés de la période des runs, ceux-là mêmes dont est parti nll_run.py : on peut ainsi comparer, événement par événement, la localisation faite sur OAR à celle faite en local. **screloc ne modifie rien dans la base** : il y lit seulement l'inventaire des stations, et écrit les résultats en XML.
INSTALLER.txt installe la configuration de screloc (screloc.cfg) si vous n'en avez pas déjà une.
En plus des lignes du plugin et du profil (comme pour scolv), ce fichier contient :
NonLinLoc.saveIntermediateOutput = false
reloc.locator = NonLinLoc
reloc.profile = 05_ubaye_1D_topo
reloc.allowManualOrigins = true

- allowManualOrigins est **indispensable** : les origines de départ sont les localisations manuelles (LOCSAT), que screloc ignore par défaut.
- saveIntermediateOutput = false évite d'écrire une grille de probabilité par événement (environ 1 Go pour un mois).

screloc lit l'inventaire dans la base : INSTALLER.txt reprend la connexion de votre configuration de scolv, puis relocalise un jour après l'autre et compte les origines produites. Deux points à connaître :

- **seiscomp exec** est nécessaire sur OAR : il charge l'environnement de SeisComP. Lancé seul, screloc ne trouve pas le plugin (« Did not find plugin locnll »).
- --origin-id-suffix .nll donne à chaque nouvelle origine l'identifiant de l'origine de départ suivi de .nll : c'est ce qui permet de retrouver l'événement. Chaque fichier de sortie contient les origines de départ et les origines relocalisées.

Le nombre d'origines relocalisées doit être celui des événements de la période (indiqué dans INSTALLER.txt, qui donne la commande pour les compter).
Différence à connaître avec le local : screloc utilise aussi les pointés que l'analyste a désactivés dans scolv (timeUsed = false), que nll_run.py écarte. C'est la principale cause d'écart entre les deux : sur novembre 2024, avec le profil 1D de l'Ubaye, 98 % des positions sont identiques à 10 m près quand les deux côtés utilisent les mêmes phases.

Etape 6 : scrtdd avec les mêmes grilles

scrtdd (relocalisation en double différence) peut utiliser les mêmes grilles que NonLinLoc, grâce au plugin **tttnll** fourni avec scrtdd : les grilles sont déclarées comme une table de temps de trajet SeisComP (interface NLLGrid), et un profil scrtdd la désigne par son nom. La localisation absolue et la double différence reposent alors sur le même modèle de vitesse.
INSTALLER.txt vérifie que le plugin est installé, puis ajoute à la configuration de scrtdd les lignes suivantes :
plugins = ${plugins}, tttnll
ttt.NLLGrid.tables = 05_ubaye_1D_topo
ttt.NLLGrid.05_ubaye_1D_topo.tablePath = /home/desmeuzl/.seiscomp/nll/05_ubaye_1D_topo/time/layer
profile.05_ubaye_1D_topo.tableType = NLLGrid
profile.05_ubaye_1D_topo.tableModel = 05_ubaye_1D_topo
profile.05_ubaye_1D_topo.regionType = RECTANGULAR
profile.05_ubaye_1D_topo.region = 44.05, 6.10, 44.96, 7.39
Le profil scrtdd ne règle que la table de temps et la région : les autres réglages gardent les valeurs par défaut de scrtdd. On l'appelle avec --profile 05_ubaye_1D_topo.
Remarques :

- tttnll retrouve la grille d'une station par sa **position**, et non par son nom : une station déplacée de plus de 10 m (maxSearchDistance) depuis le calcul des grilles n'a plus de temps de trajet.
- Il ne fournit que les premières arrivées (P et S), ce qui suffit pour nos grilles.
- Cette étape n'a pas encore été testée sur OAR (novembre 2026).

Comprendre les paramètres

Cette partie explique, ligne par ligne, les fichiers que lit NonLinLoc : ceux qu'écrit creer_profil.py, comme ceux que l'on écrit à la main (voie manuelle). On y trouve aussi les valeurs choisies par Loc_ToolBox et la raison de ces choix. Pour le détail complet de chaque paramètre et des paramètres optionnels, la référence reste le site d'Anthony Lomax (section Control File).

Le modèle de vitesse 1D : les lignes LAYER

Exemple de modèle de vitesse 1D :

```
#LAYER depth Vp_top Vp_grad Vs_top Vs_grad p_top p_grad
LAYER    0.0     5.30    0    3.01    0    2.52    0
LAYER    4.0     5.60    0    3.18    0    2.61    0
LAYER    9.0     6.20    0    3.52    0    2.78    0
LAYER    14.0    6.90    0    3.92    0    2.97    0
LAYER    19.0    7.40    0    4.20    0    3.12    0
LAYER    24.0    7.70    0    4.37    0    3.20    0
LAYER    33.0    7.90    0    4.49    0    3.26    0
LAYER    49.0    8.10    0    4.60    0    3.32    0
LAYER    66.0    8.30    0    4.72    0    3.37    0
```

#LAYER depth Vp_top Vp_grad Vs_top Vs_grad p_top p_grad
LAYER -5.0 5.30 0 3.11 0 2.39 0
LAYER 3.0 5.92 0 3.48 0 2.61 0
LAYER 27.0 6.60 0 3.88 0 2.75 0
LAYER 35.0 8.80 0 5.17 0 2.98 0

- depth : profondeur du haut de la couche, en km sous le niveau de la mer ; la couche avec le haut le plus profond s'étend jusqu'à l'infini.
- Vp_top, Vs_top : vitesses des ondes P et S en haut de la couche, en **km/s**.
- p_top : densité en haut de la couche.
- Vp_grad, Vs_grad, p_grad : gradients qui augmentent directement depuis le haut de la couche. Vous pouvez les laisser à 0 pour une vitesse constante par couche.

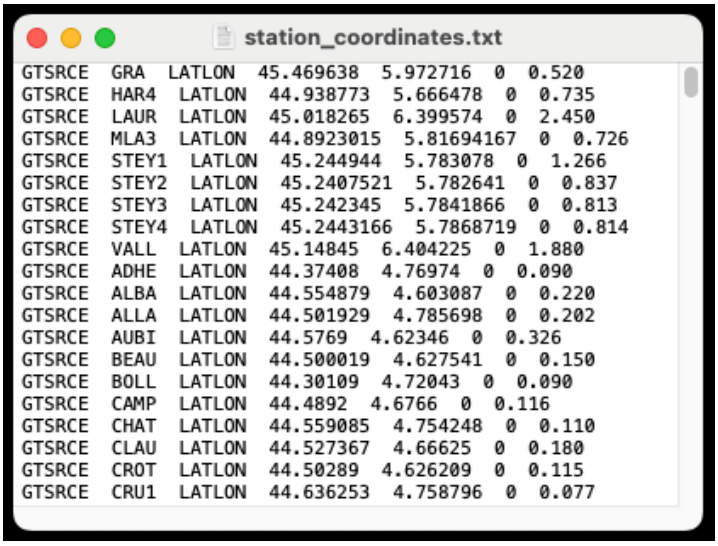
Pour calculer votre densité, il est possible d'utiliser la loi de Birch (https://planet-terre.ens-lyon.fr/ressource/noyau-fer-Terre.xml), ici on utilisera une approximation de cette loi, suggérée par Laurent Stehly : rho = vp/1000 * 0.32 + 0.77

Dans Loc_ToolBox : les modèles 1D déjà utilisés au labo (Jura, Alpes, Auvergne) sont dans le dossier modeles_vitesse/, au format « toit Vp Vs densité » ; creer_profil.py les convertit en lignes LAYER. La première couche est prolongée jusqu'au toit des grilles (-5 km), pour que les stations en altitude soient dans le modèle.

La liste des stations : GTSRCE

Il existe 4 syntaxes différentes, nous n'en verrons qu'une seule ; les autres sont décrites sur le site d'Anthony Lomax, section Control File -> Grid2Time -> GTSRCE – Source Description.
GTSRCE label LATLON latSrce longSrce zSrce elev
GTSRCE CALF LATLON 43.753 6.922 0.0 1.242

- label (texte) : nom de la source, c'est-à-dire le code de la station (ABC). C'est ce nom qui figure dans le nom des grilles de temps.
- latSrce (-90 à 90) : latitude en degrés (positive vers le nord).
- longSrce (-180 à 180) : longitude en degrés (positive vers l'est).
- zSrce : profondeur de la station dans la grille, en km (positive vers le bas).
- elev : altitude de la station au-dessus de zSrce, en km.



Dans Loc_ToolBox : nll_run.py --grilles écrit cette liste tout seul : il prend les stations pointées sur la période, avec leur position dans l'inventaire du serveur SeisComP, et ne calcule que les grilles qui manquent.

La grille de vitesse : vel2grid.in

CONTROL 1 159753
TRANS AZIMUTHAL_EQUIDIST WGS-84 44.490000 6.755000 0
VGOUT ./model/layer
VGTYPE P
VGGRID 2 401 66 0.0 0.0 -5.0 1.0 1.0 1.0 SLOW_LEN
LAYER ... (vu plus haut)

- CONTROL : niveau de détail des messages, puis graine du générateur aléatoire (voir le fichier de contrôle plus bas).
- TRANS : la projection (voir *Le système de coordonnées : 'TRANS'*). **Elle doit être strictement identique dans tous les fichiers.**
- VGOUT : le préfixe des fichiers produits. Il ne faut pas créer de dossier /layer : ce morceau du chemin sert à nommer les fichiers générés.
- VGTYPE : une ligne par phase à générer (P et S en général).
- VGGRID xNum yNum zNum xOrig yOrig zOrig dx dy dz gridType :
  - xNum = 2 active le mode 2D radial (cf. *Grille de vitesse en mode 2D radial*) ;
  - 'yNum x dy' doit couvrir la distance maximale station-événement attendue, et 'zNum x dz' la gamme de profondeurs attendues. Il est préférable d'avoir une grille un peu plus grande que plus petite, donc prévoyez une petite marge ;
  - gridType SLOW_LEN (lenteur × longueur de maille) est le type qu'attend Grid2Time.

Dans Loc_ToolBox : grille au pas --pas (en km), sur --distance-max km (400 km par défaut), de -5 km à max(60, --prof-max + 10) km de profondeur. Un fichier par phase (vel2grid_P.in, vel2grid_S.in).

Les grilles de temps : grid2time.in

CONTROL 1 159753
TRANS AZIMUTHAL_EQUIDIST WGS-84 44.490000 6.755000 0
GTFILES ./model/layer ./time/layer P
GTMODE GRID2D ANGLES_YES
GT_PLFD 1.0e-4 1
INCLUDE station_coordinates.txt

- TRANS : identique à vel2grid.in !
- GTFILES : le préfixe de la grille de vitesse, celui des grilles de temps, et la phase. C'est un paramètre obligatoire et non répétable : il faut exécuter Grid2Time deux fois, une fois avec P et une fois avec S (doit correspondre aux VGTYPE de vel2grid.in).
- GTMODE : GRID2D pour un modèle 1D (doit correspondre au 'xNum = 2' de vel2grid.in), GRID3D pour un modèle 3D. ANGLES_YES calcule aussi les angles d'incidence, utiles pour les mécanismes au foyer ; ANGLES_NO accélère le calcul si vous n'en avez pas besoin.

- GT_PLFD : réglages du calcul des temps par différences finies (méthode de Podvin et Lecomte) : précision de l'initialisation autour de la source, puis niveau des messages.
- INCLUDE : si vous avez peu de stations, il est possible de les mettre directement dans le fichier, au format GTSRCE ; sinon je recommande la méthode INCLUDE avec un fichier externe.

On obtient 2 fichiers par phase et par station avec ANGLES_NO (.time.hdr, .time.buf), 4 avec ANGLES_YES (.angle.hdr, .angle.buf en plus).
Dans Loc_ToolBox : creer_profil.py écrit ce fichier (nll_<profil>.in), sans liste de stations ; nll_run.py --grilles l'appelle pour chaque phase et chaque station qui manque.

Le fichier de contrôle de la localisation : NLL.<profil>.conf

C'est le fichier que lit NLLoc, en local comme dans SeisComP. Voici celui que creer_profil.py a écrit pour le profil de l'Ubaye (05_ubaye_1D_topo) :

```
CONTROL 1 159753
TRANS AZIMUTHAL_EQUIDIST WGS-84 44.490000 6.755000 0
LOCSIG NonLinLoc - ISTERRE SIG
LOCSEARCH OCT 24 24 15 0.001 50000 5000 0 1
LOCGRID  71 71 46 -35 -35 -5.0000  1.0000 1.0000 1.0000  PROB_DENSITY  SAVE
LOCMETH EDT_OT_WT 9999.0 4 -1 -1 1.7 -1 -1.0 1
LOCGAU 0.05 0.0
LOCPHASEID  P   P p Pn Pg P1
LOCPHASEID  S   S s Sn Sg S1
LOCQUAL2ERR 0.025 0.050 0.100 0.200 0.400 99999.9
LOCANGLES ANGLES_YES 5
LOCTOPO_SURFACE ${NLLROOT}/05_ubaye_1D_topo/topo.asc 0
```

CONTROL : niveau de messages et graine

Le premier chiffre sur la ligne du paramètre CONTROL correspond au niveau de détail du verbose (messages plus ou moins détaillés sur les actions en arrière-plan). Si vous souhaitez avoir le niveau de détail le plus haut, mettez ce chiffre à 9, très pratique pour le debugging, sinon vous pouvez le mettre à -1 pour n'avoir aucune information. Dans le cadre d'une utilisation avec scolv, je vous recommande de garder CONTROL à 9, mais avec screloc sur des milliers d'événements, il devient très intéressant de le mettre à -1 pour gagner en temps de calcul.
Le second nombre est la graine du générateur aléatoire : à graine égale, NLLoc refait exactement la même recherche. C'est ce qui permet de comparer un run local et SeisComP à la position près.

TRANS et LOCSIG

- TRANS : exactement la même que pour les grilles. profil_vers_seiscomp.py le vérifie avant de préparer l'installation.
- LOCSIG : une signature écrite dans les fichiers de résultats.

LOCGRID : la zone de recherche

LOCGRID xNum yNum zNum xOrig yOrig zOrig dx dy dz gridType saveFlag

- Elle doit contenir géographiquement toute la zone où vous attendez des événements — si un événement tombe hors de cette grille, la localisation échoue ou est aberrante. Elle doit aussi tenir dans les grilles de temps.
- xOrig, yOrig, zOrig : le coin de la grille, en km par rapport au centre de TRANS (ici -35 km : la grille est centrée).
- gridType et saveFlag : je recommande **PROB_DENSITY SAVE** si vous ne savez pas quoi mettre.

Dans Loc_ToolBox : la zone choisie, plus --marge km de chaque côté (10 km par défaut), arrondie à 5 km, de -5 km à --prof-max km, au pas de 1 km. Un toit à -5 km laisse la place aux séismes situés au-dessus du niveau de la mer, sous le relief.

LOCSEARCH : la recherche octree

LOCSEARCH OCT initNumCells_x initNumCells_y initNumCells_z minNodeSize maxNumNodes numScatter useStationsDensity stopOnMinNodeSize

NLLoc découpe d'abord la grille en cellules (24 × 24 × 15 ici), puis subdivise sans cesse les cellules les plus probables :

- initNumCells : le nombre de cellules de départ dans chaque direction. Des cellules trop grandes ou allongées donnent des épicentres « en damier » ;
- minNodeSize : la taille de cellule (km) à laquelle on s'arrête ;
- maxNumNodes : le nombre maximal de cellules examinées ;
- numScatter : le nombre de points tirés dans la densité de probabilité (le nuage autour de l'hypocentre) ;
- useStationsDensity (0) et stopOnMinNodeSize (1) : à laisser tels quels.

Dans Loc_ToolBox : des cellules de départ d'environ --cellule km de côté (3 km par défaut) ; maxNumNodes vaut 5 fois le nombre de cellules de départ, et au moins 50 000.

LOCMETH : la méthode et ses seuils

LOCMETH method maxDistStaGrid minNumberPhases maxNumberPhases minNumberSphases VpVsRatio maxNum3DGridMemory minDistStaGrid iRejectDuplicateArrivals

- EDT_OT_WT : la méthode EDT (Equal Differential Time) compare les différences de temps entre stations plutôt que les temps eux-mêmes. Elle est robuste aux pointés aberrants : un mauvais pointé est mis de côté (poids nul dans les résultats) au lieu de tirer la localisation.
- maxDistStaGrid (9999 km) : les stations plus lointaines sont ignorées ; 9999 les garde toutes.
- minNumberPhases (4) : NLL ne localise pas les événements avec moins de phases que ce seuil. C'est un des deux réglages à adapter en priorité à votre sujet d'étude.
- maxNumberPhases et minNumberSphases (-1) : pas de limite.
- VpVsRatio (1.7) : l'autre réglage à adapter. **Quand il est positif, le temps S d'une station qui a aussi un pointé P est calculé comme le temps P × VpVsRatio** : la grille S n'est alors utilisée que pour les stations sans pointé P. Avec le modèle 1D du labo (Vp/Vs = 1,70 dans toutes les couches), cela ne change rien. Avec un modèle 3D, le Vs du modèle n'est pas utilisé : pour s'en servir, mettre -1 (les grilles S sont alors lues pour toutes les phases S).
- maxNum3DGridMemory (-1) : pas de limite au nombre de grilles gardées en mémoire.
- minDistStaGrid (-1.0) : pas de distance minimale.
- iRejectDuplicateArrivals (1) : écarte les doublons (deux pointés de la même phase à la même station).

LOCGAU, LOCPHASEID, LOCQUAL2ERR, LOCANGLES, LOCTOPO_SURFACE

- LOCGAU SigmaTime CorrLen : l'erreur du modèle de vitesse, ajoutée à l'incertitude des pointés : 0,05 s, sans corrélation entre stations (longueur de corrélation 0 km).
- LOCPHASEID : les noms de phase traités comme P (P, p, Pn, Pg, P1) ou comme S. Un pointé d'un autre nom n'est pas utilisé.
- LOCQUAL2ERR : la correspondance entre la qualité d'un pointé (0 à 4) et son incertitude en secondes. Nos pointés arrivent avec une incertitude en secondes (NonLinLoc.defaultPickError quand ils n'en ont pas) : la ligne sert donc peu, mais NLLoc la demande.
- LOCANGLES ANGLES_YES 5 : calcule les angles de départ des rais (mécanismes au foyer) si vous avez choisi ANGLES_YES pour Grid2Time, sinon ANGLES_NO. Le second nombre est la qualité minimale (0 à 10) des angles retenus.
- LOCTOPO_SURFACE : la topographie (grille GMT au format texte). NLLoc exclut les hypocentres situés au-dessus du relief. Le chemin contient ${NLLROOT} : nll_run.py et profil_vers_seiscomp.py le remplacent par le vrai chemin, en local et sur OAR.

Dans Loc_ToolBox : LOCMETH, LOCGAU, LOCPHASEID, LOCQUAL2ERR et LOCANGLES sont les mêmes pour tous les profils (les réglages standards ci-dessus) ; LOCSEARCH, LOCGRID et TRANS dépendent de la zone ; LOCTOPO_SURFACE n'est écrite qu'avec la topographie.

La voie manuelle

Cette partie décrit ce que font les outils de Loc_ToolBox, avec les programmes d'Anthony Lomax. Elle reste utile pour comprendre les fichiers produits, ou pour un cas que les outils ne couvrent pas. Le détail des paramètres est dans *Comprendre les paramètres*.

Ce dont nous avons besoin et qui n'est pas fourni par NonLinLoc

1. Les algorithmes mis à disposition par Anthony Lomax pour générer les grilles de vitesse nécessaires au bon fonctionnement de NLL
2. Un modèle de vitesse ondes P et S (lignes LAYER en 1D)
3. La liste des stations avec leurs positions respectives (lignes GTSRCE), à sauvegarder dans un fichier .txt qu'on pourra nommer station_coordinates.txt

Obtenir les algorithmes du package NonLinLoc - Installation

Le plus simple est python installer_nll.py (Loc_ToolBox), qui télécharge NonLinLoc depuis https://github.com/alomax/NonLinLoc, le compile et range les programmes dans le dossier de travail.
À la main : aller sur le GitHub de NLL, télécharger le répertoire, et suivre le README qui explique comment compiler les programmes, qui seront ensuite disponibles dans src/bin/. Ceux qui nous intéressent dans ce document sont Vel2Grid (et/ou Vel2Grid3D) et Grid2Time, donc assurez-vous bien qu'ils soient là.

Obtention des grilles de temps de trajet avec un modèle 1D

Etape 1 Vel2Grid : la grille de vitesse

Écrivez vel2grid.in (voir *La grille de vitesse : vel2grid.in*), placez-vous dans son répertoire (c'est-à-dire celui du projet en cours) et faites ensuite appel à Vel2Grid :
Vel2Grid vel2grid.in
Vous devriez retrouver les 4 documents ci-dessous dans votre sous répertoire /model.

```
sysop@ist-sc3test-sig:~/nll$ cd model/
sysop@ist-sc3test-sig:~/nll/model$ ls -l
total 680
-rw-r--r-- 1 sysop sysop 340048 Oct 27 11:47 layer.P.mod.buf
-rw-r--r-- 1 sysop sysop    154 Oct 27 11:47 layer.P.mod.hdr
-rw-r--r-- 1 sysop sysop 340048 Oct 27 11:47 layer.S.mod.buf
-rw-r--r-- 1 sysop sysop    154 Oct 27 11:47 layer.S.mod.hdr
```

Etape 2 Grid2Time : les grilles de temps de trajets

Écrivez grid2time.in (voir *Les grilles de temps : grid2time.in*), puis exécutez-le une fois pour P et une fois pour S (en changeant la phase de GTFILES) :
Grid2Time grid2time.in
Vérifiez bien qu'il n'y a pas eu de message d'erreur et que toutes les grilles de temps de trajets ont été générées.

Obtention des grilles de temps de trajet avec le modèle de vitesse 3D de Jean Virieux

Si votre modèle de vitesse 3D est déjà au format de sortie d'un des programmes suivants : **SimulPS**, **Simul2000** ou **FDTomo**, alors je vous conseille de vous rendre dans la section Vel2Grid3D du site internet de monsieur Lomax et de suivre les indications. Dans le cas contraire, il est recommandé (explications disponibles sous la réponse de la première question de la FAQ sur le site internet) de développer vous-même un script qui permettra de reconstruire directement le modèle de vitesse au bon format pour procéder ensuite à la génération des grilles de temps de trajets. Vous pouvez trouver toutes les informations des formats cibles dans la partie Formats 3D Grid Files du site internet.

Etape 1 Variante à Vel2Grid : Mettre le modèle de vitesse au bon format

Le modèle de vitesse de Jean Virieux (wet_new.asc) est au format lon, lat, depth, vp (m/s) et vs (m/s), avec un échantillonnage régulier de 0,04° en lon et lat et de 1000 mètres en profondeur.
Le plus simple est creer_profil.py --modele-3d, qui découpe une tuile autour de la zone et la convertit au format NLL. Il s'appuie sur deux scripts de Loc_ToolBox, utilisables seuls :

- segment_from_zones.py : extrait la (ou les) partie du modèle de vitesse nécessaire pour l'étude, d'après la section segment.zones de config.yaml ;
- nll_pipeline.py : à partir du résultat du premier script, écrit le modèle au format de sortie de Vel2Grid (grille 3D), prêt pour Grid2Time, ainsi que la topographie.

Les scripts sont utilisables avec un autre modèle de vitesse si vous modifiez la partie lecture du modèle de vitesse. Rendez-vous sur le README de Loc_ToolBox (https://github.com/LuDesmeuzes/Loc_ToolBox) pour des explications complètes.

Etape 2 Grid2Time : Calculer les grilles de temps de trajets

Comme en 1D, avec GTMODE GRID3D ANGLES_YES : une exécution pour les phases P, une pour les phases S.
Grid2Time grid2time.in

Préparer le fichier de contrôle pour NLL sur SeisComP

Écrivez NLL.<profil>.conf sur le modèle de *Le fichier de contrôle de la localisation*, en reprenant la ligne TRANS des grilles et une LOCGRID qui couvre vos séismes.
Pour l'installer sur OAR à la main, on recrée la structure de l'étape 2 (fichier de contrôle et dossier time/ dans .seiscomp/nll/<profil>/), puis on ajoute à la configuration de scolv les lignes de l'étape 3.